

Soporte excepciones en un procesador segmentado

Hemos añadido al *MIPS* segmentado de clase soporte para gestionar *IRQs* (excepciones externas, no vinculadas a las instrucciones del pipeline). Este soporte incluye un registro de estado (*SR*), un registro para almacenar la dirección de retorno de la *IRQ* (*IRQ_LR*) y algunos multiplexores y puertas lógicas. Los multiplexores, y las señales de control de los registros *PC*, *SR* e *IRQ_LR* todavía no están definidos en la especificación de vhdL del procesador, y vuestro trabajo es diseñar esa parte. En vuestros diseños debéis utilizar las siguientes señales del nuevo *MIPS*:

- *IRQ*: línea de petición externa de interrupción.
- *IRQ_disable*: se pone a '1' cuando se deshabilitan las interrupciones.
- *IRQ_accepted*: se pone a '1' en el ciclo en el que se acepta la interrupción.
- *RTE*: indica que la instrucción en Decodificación es una *RTE* (*Retorno Tratamiento de Excepción*).
- *Valid_ex* y *valid_id*: bits de validez de las instrucciones en *EX* e *ID*.
- *Parar_ID*: indica que la instrucción en *ID* no puede continuar su ejecución debido a un riesgo de datos y por tanto hay que parar *ID* e *IF*.
- *Parar_MIPS*: se activa cuando hay riesgos estructurales en la etapa de Memoria. En esos casos **se detiene la ejecución por completo** hasta que el riesgo desaparece.

Ayuda: en el lenguaje de descripción de hardware VHDL se puede diseñar un multiplexor y su lógica de control a la vez mediante la sentencia `when ... else`. Ejemplo:

```
Salida_mux <=      Entrada_1 when condicion_1 else
                  Entrada_2 when condicion_2 else
                  Entrada_3;
```

Os recomendamos leer el guión del proyecto y los fuentes del MIPS en vhdL.

1. Gestión de la IRQ

Vamos a describir la gestión de las *IRQs* en este procesador.

1. Cuando llegue una *IRQ* (es decir, cuando *IRQ*='1'), esta se procesará siempre que las *IRQs* no estén desactivadas (*IRQ_disable*='1') y el procesador no esté detenido (*Parar_MIPS*='1').
2. Para gestionar una interrupción, el procesador saltará de forma automática a la dirección: `0x00000004` y se actualizará el registro de estado deshabilitando las interrupciones. También almacenará la dirección de retorno en el registro *LR_IRQ*.

3. Cuando se ejecute una instrucción RTE se volverá a la ejecución normal. Para ello se cargará en PC, la dirección de retorno, y se actualizará el registro de estado habilitando las interrupciones.

Diseñad en VHDL la lógica combinacional que asigna el valor a la señal IRQ_accepted

En los tres próximos apartados veremos los detalles del sistema que debéis diseñar.

2. Gestión del PC

Como se ve en la figura, tenemos dos señales para gestionar el PC: *PC_src* que elige qué dirección se carga, y *Load_PC* que indica si hay que cargar algo o no.

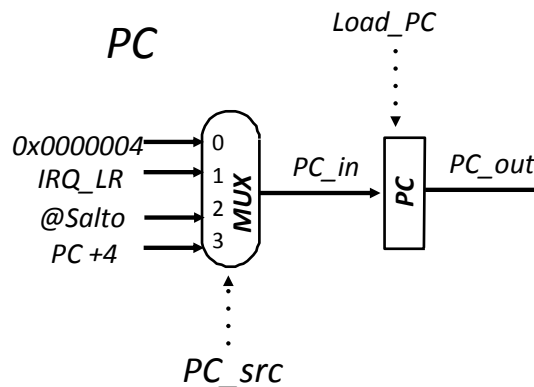


Figura 1: gestión del registro PC en un MIPS con soporte para IRQ.

PC_src: En el MIPS visto en clase de teoría hay un mux que elige entre *PC+4* o la dirección de salto. Para gestionar interrupciones (IRQs) vamos a añadir dos opciones más:

- *0x00000004*: es la dirección asignada a las IRQs en la tabla de excepciones del sistema. En ella debe estar la instrucción que salta a la rutina de tratamiento de las interrupciones (RTI). Cuando llega una IRQ, si las IRQs están habilitadas, se debe saltar a esta dirección de forma automática.
- *IRQ_LR*: es la dirección de retorno de la IRQ. Se debe cargar en PC cuando se ejecuta una instrucción RTE.

Load_PC: en un segmentado cada ciclo empieza la ejecución de una nueva instrucción a no ser que se reciba la señal de parar: *Parar_ID* o *Parar_Mips*. Si *Parar_Mips* está activado, para simplificar la gestión, detendremos el procesador por completo. Eso incluye no procesar la interrupción recibida. La procesaremos cuando el procesador deje de estar parado.

Por otro lado, *Parar_ID* es una señal generada por la UC cuando detecta que hay riesgos de datos en la instrucción que está en ID. En esa situación da la orden de parar la instrucción en ID (y por tanto también la que está en IF). **Pregunta:** ¿qué hacemos si llega una IRQ cuando las IRQs están habilitadas y a la vez se activa *Parar_ID*? ¿Activamos o no activamos el *Load_PC*?

Diseñad en VHDL el multiplexor de la figura y la lógica combinacional que genera *PC_src* y *Load_PC*.

3. Dirección de retorno de la IRQ

Cuando se recibe una *IRQ* habilitada se guarda la dirección de retorno en un registro específico que vamos a llamar *IRQ_LR*. De esta forma al terminar la *IRQ* regresaremos a la instrucción que se estaba ejecutando antes de la *IRQ*. Pero, ¿qué instrucción es esa? En un procesador segmentado hay cinco instrucciones en ejecución simultáneamente. Hay que tomar decisiones para cada una de ellas. En la solución que vamos a usar se realizan las siguientes acciones al aceptar una *IRQ*:

- Se permite que las instrucciones que estaban en las etapas *MEM* y *WB* continúen su ejecución. Permitir que termine la instrucción en *WB* es lógico porque ya está terminando (a mitad de ciclo se escribirá su resultado en el banco de registros). En cuanto a la instrucción en *MEM*, cuando hay jerarquía de memoria, los accesos a memoria pueden tardar muchos ciclos, y es posible que la instrucción en *MEM* esté por ejemplo a mitad de una escritura. Interrumpir su ejecución dejaría al sistema temporalmente en un estado inestable. Esto lo entenderéis mejor cuando estudiemos la jerarquía de memoria.
- Se invalidan las instrucciones que estaban en las etapas *EX*, *ID* e *IF*. El objetivo es dar máxima prioridad a la ejecución de la *IRQ*, evitando retrasos producidos por detenciones generadas por estas instrucciones.

La pregunta es ¿Qué dirección hay que almacenar en el *IRQ_LR*? Como se ve en la figura hay tres opciones: *PC_EX*, *PC_ID*, que son las direcciones de las instrucciones que hay en esas etapas, o bien el *PC*. La respuesta es sencilla: **hay que volver a la primera instrucción válida de las tres que se anularon**. Las instrucciones en una etapa pueden no ser válidas debido a los riesgos de datos o control. Por un lado, si en *ID* se detecta un riesgo de datos, se detiene la ejecución en *ID* y se envía una instrucción inválida a *EX* (no hay ninguna válida que mandar). Por otro lado, si en *ID* hay una instrucción de salto y hay que saltar (salto tomado), en *IF* estará normalmente la instrucción del salto no tomado, y por tanto hay que intervenir para que cuando pase al ciclo siguiente quede invalidada. En un sistema más complejo, leer una instrucción en *IF* podría tardar varios ciclos, durante los cuales se enviarían instrucciones inválidas hacia adelante. Pero **hay una excepción** a la regla general: Si la instrucción en *EX* es una RTE (de una interrupción anterior), no se elegirá como dirección de retorno. **Intentad explicar por qué.**

Escribid una secuencia de dos instrucciones tales que si se acepta una instrucción en un determinado ciclo (indicad cuál), la dirección de retorno sea *PC_ID* en lugar de *PC_EX*.

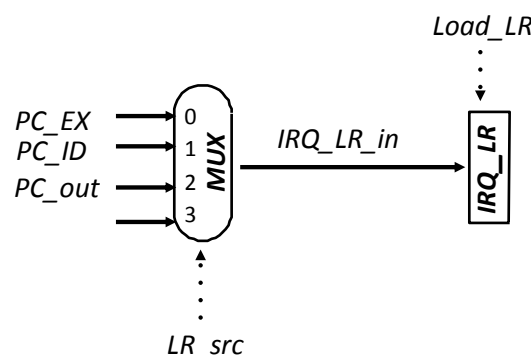


Figura 2. Gestión de la dirección de retorno de la IRQ.

Diseñad el multiplexor de la figura con una sentencia `when .... else` y la lógica combinacional que genera `Load_LR` en `vhdl`.

4. Registro de estado (SR, Status Register)

El registro de estado en esta implementación tiene sólo dos bits:

- El bit más significativo (*IRQ_disable*) permite deshabilitar (valor 1) o habilitar las excepciones.
- El bit menos significativo (*Mode*) informa si estamos en modo excepción (valor 1) o estamos en modo normal.

Como se puede ver en la figura, en esta implementación van los dos a la par. Cuando estamos en modo excepción se deshabilitan las *IRQ* (valor 11), y cuando volvemos a modo normal se vuelven a habilitar (valor 00).

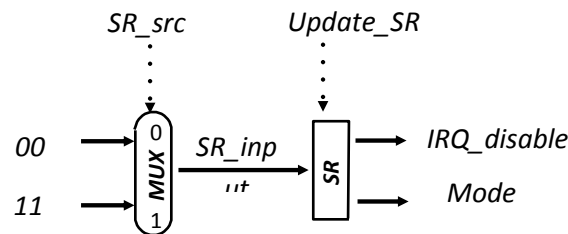


Figura 3. Gestión del registro de estado

Diseñad en VHDL el multiplexor de la figura con una sentencia `when .... else` y la lógica combinacional que genera `SR_src` y `Update_SR`.